3D Reconstruction into Minecraft using SfM

Tyler Mitts
Dept. of Electrical Engineering and
Computer Science,
University of Missouri
Columbia, MO 65211, USA
tmitts0206@gmail.com

***Abstract*—Using a SfM transformer model to reconstruct a sequence of images into 3D reconstruction to later turn in a Minecraft schematic.**

Keywords—SfM, VGGT, 3D Reconstruction, Minecraft Schematic

I. Introduction: Motivation and Background

In previous work, I have worked to design a pipeline to convert 2D images into 3D Minecraft using depth estimation. The motivation for this paper is to expand on that idea and construct a pipeline that fully encompasses the 3D object in images by using SfM to create a 3D reconstruction using a sequence of images. This will return a much more accurate 3D representation then the use of one image and depth estimation. This would allow for artists and creative Minecraft users to fully bring real-life into game by using this pipeline to turn video into a Minecraft schematic. For this paper, a dataset called "Temple of the Dioskouroi" will be used, which is a set of 312 images of different camera angles of this temple within Rome.

II. Image Processing Modules and Pipelines

A. Modules Used

OpenCv, PyTorch, Scikit-Image, NumPy, Matplotlib, Open3D, OS.

B. Pipeline Used

A video or sequence of images are inputted, where a specific number of frames are chosen to actually be processed through the pipeline. The images are then resized to 518x518 pixels as required by the VGGT model. These images are then sent through VGGT to extract depth maps and camera pose estimations. Using this information, VGGT generates a dense point cloud to reconstruct the image information into 3D structure. Filtering is then used on the point cloud to threshold and reduce the noise within the point cloud. The point cloud is then segmented into a grid of voxels, transforming the information so that it would work converting it to Minecraft. Post processing then occurs to fill in the holes of the build and ensure that it outputs the real structure of objects and now just spaced out blocks. The colors of these voxels are converted to Minecraft textures, where then the voxel position information and textures can be used to create a Minecraft Litematica schematic.

III. Approach: Algorithms and Module Implementation Details

The following pipeline will be used on a sequence of images of the Temple seen in figure 1. A sample of 30 images will be used as input into the VGGT model.



Figure 1. Image from temple dataset.

For 3D reconstruction, the VGGT model is used to create a dense point cloud [2]. This model was chosen as it performs the functionality of SfM through a single forward pass from the transformer, allowing it to be done fast and efficiently. VGGT is able to accurately reconstruct 3D structure from a sequence of images as it was trained on a wide variety of datasets of both real and synthetic data to teach the model camera motion and 3D structure. After inputting a sequence of images into the model, it generates depth maps of each image, a dense point cloud, and the estimated camera parameters.



Figure 2. Dense point cloud reconstruction.

Since the temple dataset used has ground truth data regarding the bounding box for the temple as well as the parameters for the cameras, using the information gained from VGGT can be compared to test the accuracy of the model. After computing the original point cloud, it is then filtered to reduce the noise and false positives. This is done by filtering by confidence, for each point, if the confidence is higher than the set threshold, which in this case we use the threshold value t=0.92, then the point is kept, otherwise it is discarded.

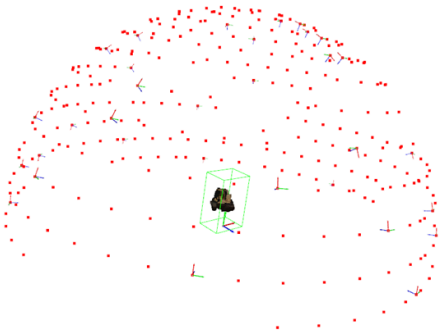


Figure 3. Temple 3D reconstruction within ground truth bounding box and cameras.

Error Type	Error Average
Rotation	154.27°
Translation	1.3049m

Figure 4. VGGT vs. ground truth camera placement error.

X	Y	Z
0.6511	0.3639	1.0

Figure 5. VGGT-to-ground truth bounding box ratio for Temple



Figure 6. 3D reconstruction after point cloud after grid cell segmentation and voxelization.

After generating the point cloud 3D reconstruction, the points are segmented into grid cells so that they can be voxelized, turned from points to cubes, for the sake of converting them to Minecraft blocks. This is done by creating a 3D grid, which in this case uses a voxel size of 1. The voxelized version of the point cloud is seen in figure 6. Each of the points in the cloud are assigned to a grid cell, then for a non-empty grid cell it takes the average color of all the points within the cell and assigns that color to the cell. To ensure the point cloud generates a solid structure comparable to the real physical structure, a hole filling algorithm is used to fill in gaps between densely placed voxels. The hole filling algorithm checks the neighbors of each empty voxel, and if the average of the neighbors are filled and above the threshold, then that empty voxel also gets filled.

Now the colors of each grid cell needs to be matched to a Minecraft block texture for converting it to the game. The average color of both the cells and all of the Minecraft textures are taken, allowing for them to be compared seamlessly. The RGB values of the cells are then matched to the closest RGB values of Minecraft textures by using scikit-learn's nearest neighbor model, specifically the ball-tree algorithm [1]. This algorithm works well for matching 3D data as it allows for quick neighbor searching by splitting the space into hierarchical spheres. This is done by setting the two furthest points (Minecraft RGB values) and using them as pivots, assigning the other points to the closest pivot sphere. This is repeated for each subgroup until the group becomes less than or equal to the max group size. This creates a sort of algorithmic tree for the new point (cell RGB value) to traverse efficiently, having a time complexity of O(nlog(n)) for initial construction and O(n) for query.

Once the cells are mapped to Minecraft blocks, they are then placed into a 3D coordinate system based on their (x,y,z) values determined by the image coordinates and depth of the cluster. Using this information, the Litematica file can be made, a file that uses the Litematica mod in Minecraft to automatically place every block.

IV. Experimental Results and Discussion

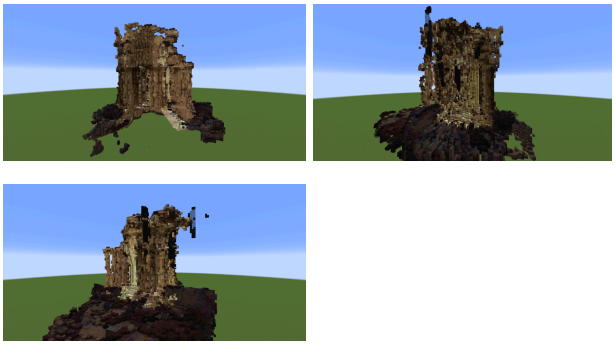


Figure 7. Results of Temple reconstruction into Minecraft.

As seen in figure 7, the pipeline was successfully able to recreate the general structure of the temple, albeit there are many inaccuracies along with it. Despite having a high threshold for the confidence filtering, there are still parts of the build that contain noise and decrease the accuracy between the real structure and the in-game structure. The model also struggled with accurately depicting the details within the columns, with some of them getting merged within the cloud point reconstruction. Depending on the number of images used for the reconstruction, the level of accuracy changed, where giving the model a specific amount of information gave the best results. Giving the model all 312 images from the dataset, the accuracy went way down and the structure wasn't even able to be recognized as there was a ton of noise. With testing, it was found that using 15-30 frames gave reasonable results.

Looking at figures 3-5, a comparison between the ground truth values given from the dataset can be made with the values generated by VGGT. In figure 4, the cameras generated by VGGT were not very accurate when comparing them to the ground truth values for those cameras, as the average rotation error is 154.27°, showing that the estimated angle of the cameras were quite inaccurate, leading to a worse accuracy with the 3D reconstruction. The model also struggled in scaling the build accurately, as the x and y ratios to the ground truth were far off when looking at figure 5.

Overall, the pipeline was able to establish the general structure and color of the dataset, though failed to recreate it in great detail, as noise and inaccuracies in the model caused the end result to look quite messy. Future work towards this would be fine-tuning the model so that it generates better initial point clouds, and find the optimal number of frames to use in specific datasets. Additionally, making it more robust so that it would be able to work for all datasets, allowing for any decent video to be taken and accurately reconstructed into Minecraft, expanding the creative possibilities with this pipeline.

References

[1] S. Omohundro, “Five Balltree Construction Algorithms,” Sep. 20, 1989. https://www.academia.edu/70430197/Five_Balltree_Construction_Algorithms

[2] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "VGGT: Visual Geometry Grounded Transformer," arXiv:2503.11651, Mar. 2025.